

iMessage Integration: Feasibility, MessageEnvelope & Simulator

Sep 27, 2026 · @re

Summary and recommendation

iMessage is feasible this semester: OpenClaw already ships an official iMessage channel, so our job is to deploy, wire and test it rather than build one from scratch. The hard constraint is hardware: the channel needs a Mac that is signed in to Messages. A real-Mac test (see Deployment log) got through installation and permissions; iMessage activation for the new bot Apple ID is the one open blocker.

Recommended path

1. **Prototype on our own OpenClaw.** Run the official iMessage plugin on Rouye's Mac, under a separate macOS user with a dedicated Apple ID, text and attachments only, no System Integrity Protection changes.
2. **Freeze MessageEnvelope v0.1** (Section 5) as the contract between the iMessage channel and the MCP / workflow workstreams.
3. **Build the Simulator** (Section 6) so teammates without a Mac can develop and test against realistic iMessage traffic.
4. **Keep a hosted relay as the fallback** if Virtual Gold cannot host a Mac, and let the client decide on the data-privacy trade-off.

Deliverable	Where	Status
iMessage technical options comparison	Section 3	Draft
MessageEnvelope draft	Section 5	Draft v0.1
Simulator design	Section 6	Draft

Key finding: OpenClaw already has an iMessage channel

The current OpenClaw docs list an official iMessage plugin (`@openclaw/imessage`). The Gateway spawns the `imsg` CLI and talks to it over line-delimited JSON-RPC on stdio, with no separate daemon or port ([OpenClaw iMessage docs](#)). BlueBubbles support has been removed; `imsg` is now the only supported path ([announcement](#)).

What it needs ([setup docs](#)):

- A Mac on macOS 14 Sonoma or later (`imsg` 0.15.9 refuses to install on macOS 13, tested 2026-09-29), signed in to Messages, running `imsg` .
- Full Disk Access (to read the Messages database `chat.db`) and Automation permission (to send through Messages.app).
- Basic text and media send/receive work as-is. Replies, tapbacks, edits, polls and group actions need the "private API", which requires disabling System Integrity Protection. We should not ask an enterprise client to do that.

Conflict to raise with Aarvin. On 9/21 Aarvin said OpenClaw has no iMessage capability of its own. The docs say otherwise. Possible explanations, which we cannot determine from here: VG runs an older OpenClaw release, VG's gateway runs on a Linux VM where the plugin cannot work without a Mac, or VG routes channels through its own adapters (as it does for Gmail) instead of OpenClaw's built-in channels.

Integration options compared

Option A is the fastest route to a working prototype; Option C is the best fit if VG's identity and permission logic lives in its own channel adapters. Aarvin's two ideas from 9/21 map onto A ("spin up our own OpenClaw with iMessage") and C ("an adapter like the Gmail intake").

Option	How it works	Needs	Main advantage	Main drawback
A. Official plugin, Gateway on the Mac	OpenClaw Gateway and <code>imsg</code> run on the same signed-in Mac	1 Mac, 1 Apple ID, Full Disk Access + Automation	Maintained by OpenClaw; attachments, pairing, dedupe and restart recovery built in	VG's core agent runs on a VM, so it would need a Mac-hosted gateway

Option	How it works	Needs	Main advantage	Main drawback
A2. Official plugin, remote Mac over SSH	Gateway stays on VG's VM; <code>cliPath</code> points to an SSH wrapper that runs <code>imsg</code> on a Mac	Mac reachable over SSH (e.g. Tailscale), strict host keys	Keeps the existing VG host	Docs warn sends can fail with AppleEvents error -1743 when Automation is granted to the SSH process
B. Hosted iMessage relay	A vendor (e.g. Linq) runs the Apple hardware; a channel plugin receives webhooks and sends via API	Vendor account and phone line	No Mac for us; community OpenClaw plugins exist	Every executive message passes through a third party; vendor claims unverified
C. Our own adapter service	A small service on a Mac reads <code>imsg</code> , emits MessageEnvelope to a VG endpoint, and delivers replies	1 Mac + our code	Mirrors VG's Gmail intake; independent of OpenClaw version; VG keeps its IAM layer in front	We own reliability, dedupe and recovery that the plugin already solves
D. Apple Messages for Business	Apple's official business channel via a messaging service provider	Apple business registration + provider	Fully sanctioned by Apple	Customer-initiated, business-branded chats; registration effort does not fit one semester

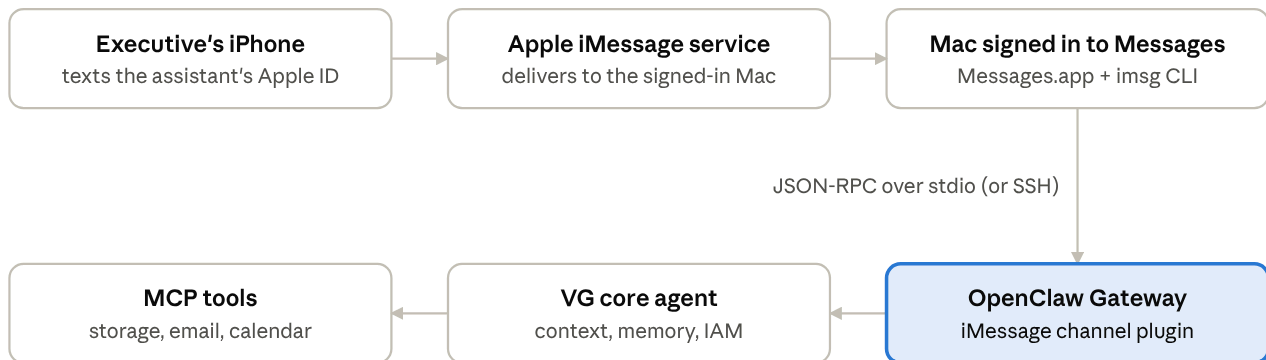
Recommendation. Prototype with A this week. Decide between A2 and C once Aarvin confirms where VG's identity layer runs (question 1 in Section 7). Keep B as the fallback only if the client accepts a third-party processor. Drop D for this semester.

How OpenClaw channels work

A channel plugin translates one messaging platform into OpenClaw's shared message model; the core agent never sees platform details. According to the [plugin SDK guide](#), a channel plugin owns config, DM security and allowlists, pairing, session grammar, outbound sending and threading, while core owns the shared `message` tool, prompt wiring and dispatch.

An iMessage request reaches the VG agent through a Mac

Highlighted: the part this workstream deploys and tests



Replies travel back along the same path; the Gateway handles dedupe and restart recovery.

iMessage request path · 6 components

What this means for the VG assistant

- **Two identity gates.** OpenClaw's iMessage DMs default to pairing mode, with an `allowFrom` list. That only decides who may talk to the bot. VG's principal / delegate / colleague / external roles still have to come from VG's own IAM layer.
- **Attachments are off by default.** `channels.imessage.includeAttachments` must be set to `true`, or attachment-only messages are dropped before reaching the agent ([media docs](#)).
- **Limits to design around.** Outbound media defaults to 16 MB (`mediaMaxMb`); outbound text is chunked at 4,000 characters (`textChunkLimit`); Markdown styling is converted to native iMessage styling.
- **Stable addressing.** The docs recommend `chat_id:<n>` targets over raw phone numbers or emails for routing replies.
- **Reliability is built in.** After a restart, the channel replays missed messages and dedupes them, which Option C would have to rebuild.

MessageEnvelope v0.1 (draft)

MessageEnvelope is the one shape every channel hands to the rest of the system, so iMessage and Gmail look the same to the agent and the MCP tools. v0.1 keeps every field already in the team PRD (`message_id`, `correlation_id`, `channel`, `conversation_id`, `actor_id`, `principal_id`, `actor_role`, `text`, `attachments`, `received_at`) and adds what iMessage needs: the platform message ID for dedupe, a reply target, and flags for echo and group handling.

Design rules

- The channel **reports**, it does not **decide**: it fills `sender.handle` and leaves `actor_role` as `unresolved`. VG's IAM assigns principal / delegate / colleague / external.
- Attachments travel **by reference**, never inline, so logs and envelopes never carry file bytes.
- One envelope per inbound message; retries reuse the same `message_id`.

Inbound envelope (example)

```
{
  "envelope_version": "0.1",
  "message_id": "msg_01J9Q4K2",
  "correlation_id": "corr_01J9Q4K2",
  "channel": "imessage",
  "channel_message_id": "imsg-guid-4F2C9A10",
  "conversation": {
    "conversation_id": "imessage:chat_id:42",
    "type": "direct",
    "reply_target": "chat_id:42"
  },
  "sender": {
    "handle": "+14125550123",
    "handle_type": "phone",
    "actor_id": null,
    "actor_role": "unresolved"
  },
  "principal_id": "exec_001",
  "text": "Can you check this invoice and tell me if anything looks off?",
  "attachments": [
    {
      "attachment_id": "att_01J9Q4K3",
      "filename": "invoice_acme_0923.pdf",
```

```

    "media_type": "application/pdf",
    "size_bytes": 182044,
    "sha256": "<hex digest>",
    "content_ref": "local://attachments/att_01J9Q4K3",
    "validation": "accepted",
    "reject_reason": null,
    "redaction_status": "pending"
  }
],
"in_reply_to": null,
"flags": { "is_from_assistant": false, "is_group": false },
"sent_at": "2026-10-01T14:02:11Z",
"received_at": "2026-10-01T14:02:13Z"
}

```

Reply envelope (outbound)

```

{
  "correlation_id": "corr_01J9Q4K2",
  "in_reply_to": "msg_01J9Q4K2",
  "channel": "imessage",
  "reply_target": "chat_id:42",
  "kind": "result",
  "text": "I found 2 issues in this invoice: the total does not match the
line items, and the invoice number repeats one from August.",
  "attachments": []
}

```

`kind` is one of `ack`, `result`, `confirmation_request`, `clarification`, `error`, so the PRD's confirm-before-send flow maps onto one field.

Validation rules (proposed values, to confirm with the team)

Rule	v0.1 behavior
Dedupe	Key = <code>channel</code> + <code>channel_message_id</code> ; drop repeats seen in the last 24 hours
Echo loop	Drop messages sent by the assistant's own Apple ID (<code>is_from_assistant: true</code>)

Rule	v0.1 behavior
Attachment types	Allow PDF, PNG, JPEG, HEIC, DOCX, XLSX, CSV, TXT; reject others with a <code>reject_reason</code> and a reply to the user
Attachment size	Reject above 16 MB, matching OpenClaw's outbound default
Unknown sender	Still build the envelope with <code>actor_role: unresolved</code> ; VG policy decides
Group chats	Out of MVP scope: set <code>is_group: true</code> , reply asking the user to message directly
Logging	Log IDs, sizes, hashes and status only; never message text or file contents

iMessage Simulator design

The Simulator lets anyone on the team push realistic iMessage traffic through the system without a Mac, and replays the same scenarios as automated tests. It uses only synthetic data: fake numbers in the 555 range and made-up invoices, never client data.

Three modes, built in this order

Mode	What it does	Why
1. Envelope mode (MVP)	Reads a scenario file, builds MessageEnvelopes, posts them to the mock assistant / MCP endpoint, captures ReplyEnvelopes and checks them	Unblocks the MCP and workflow workstreams right away
2. Raw-event mode	Generates iMessage-shaped raw events (GUID, handle, <code>chat_id</code> , attachment paths) and runs them through our normalizer	Tests the normalizer exactly as it will run on real <code>imgsg</code> output

Mode	What it does	Why
3. Fake <code>imsg</code> (stretch)	A stub speaking the same line-delimited JSON-RPC as <code>imsg rpc</code> , so an unmodified OpenClaw Gateway runs on Linux	Would let CI test the real plugin; feasibility unknown until we read the RPC reference

Scenario file (example)

```

id: TC-02-invoice-pdf
sender: "+14125550123"
messages:
  - text: "Can you check this invoice?"
    attachments: [fixtures/invoice_acme_0923.pdf]
expect:
  envelope_count: 1
  attachments[0].validation: accepted
  attachments[0].redaction_status: pending
  reply.kind: result
  reply.reply_target: chat_id:42

```

Test cases

ID	Scenario	Expected result
TC-01	Plain text request	One envelope; reply goes back to the same <code>chat_id</code>
TC-02	Text + PDF invoice	Attachment accepted, hash set, redaction pending
TC-03	Unsupported file (.zip)	Rejected with a reason; user gets a clear reply
TC-04	Attachment over 16 MB	Rejected before reaching the agent
TC-05	Same message delivered twice	Exactly one envelope
TC-06	Assistant's own reply echoed back	Dropped, no loop

ID	Scenario	Expected result
TC-07	Unknown sender	Envelope with <code>actor_role: unresolved</code>
TC-08	Group chat message	<code>is_group: true</code> ; reply asks the user to message directly
TC-09	Command, then a URL a few seconds later	Behaves like OpenClaw's split-send coalescing (to verify)
TC-10	HEIC photo from an iPhone	Accepted; flag whether downstream tools need JPEG

Build plan. Week 1: envelope mode with TC-01 to TC-05. Week 2: raw-event mode and TC-06 to TC-10, wired into the team CI.

Deployment log: real Mac test (Sep 28–30)

The channel installs on a real Mac, but the new bot Apple ID cannot yet activate iMessage, so no messages reach the database. Everything else in the setup path works. Tested on a MacBook Pro (Apple silicon) with a dedicated standard macOS user `vgbot` and a new Apple ID `vgcapstonebot@gmail.com`.

Step	What happened	Fix / status
Install <code>imsg</code> on macOS 13.5 Ventura	Homebrew refused: "does not run on macOS versions older than Sonoma"	Upgraded to macOS 27.0.1
Install <code>imsg</code> after the upgrade	Homebrew refused to build: Xcode 15.2 and Command Line Tools too old	Removed Xcode.app, installed Command Line Tools for Xcode 27.0; <code>imsg</code> 0.15.9 installed
Install tools as <code>vgbot</code>	A standard user cannot use Homebrew	Install <code>imsg</code> from the admin account; it is available to all users
Read the Messages database	<code>imsg chats</code> failed with "unable to open database file (code 14)"	Granted Full Disk Access to Terminal in the <code>vgbot</code> session; permission now works

Step	What happened	Fix / status
Sign in to iMessage (macOS 13.5)	"An unknown error occurred"; the Apple ID email was not yet verified	Verified the email
Sign in to iMessage (macOS 27.0.1)	After "Sign In", the window returns to the sign-in screen. The Messages process keeps running (same PID), so it is not a crash. No <code>chat.db</code> is ever created	Open blocker
Sign in to FaceTime, same Apple ID	Works	Account itself is healthy

Evidence on the blocker

- No `Messages` crash report exists; the only reports were unrelated system faults triggered by a remote-desktop app. (S)
- System logs for the `vgbot` identity service show the email being registered and a re-registration timer set about 26 days out, with no error lines. (S for the log lines)
- Interpretation: Apple is likely restricting iMessage activation for a brand-new Apple ID while other Apple services work. (B: fits every observation, but we cannot confirm it from the Mac)

Next steps

1. Retry iMessage sign-in after 24–48 hours, without repeated attempts in between.
2. If it still fails, contact Apple Support with the exact symptom above.
3. Fallback: use an older, established Apple ID dedicated to the bot.

What this means for Virtual Gold. A production iMessage channel needs a provisioning step we did not expect: a new assistant Apple ID may not activate iMessage on day one. Budget lead time, or reuse an established account.

Useful commands we found

- `log show` needs admin rights; reproduce in `vgbot`, then run `sudo log show ...` from the admin account (system logs are machine-wide).
- `ls ~/Library/Messages/` is the quickest check: no `chat.db` means iMessage is not signed in, whatever the UI shows.

Risks and open questions for Virtual Gold

The biggest open question is where VG's identity layer runs, because it decides between Option A2 and Option C.

Questions to send before the next meeting

1. Does VG's identity / permission logic run inside OpenClaw, or in your own channel adapters (like the Gmail intake)? If it is in the adapter, OpenClaw's built-in iMessage channel would bypass it.
2. Which OpenClaw version does VG08 run, and does its gateway run on a Linux VM or a Mac?
3. Can VG provide, or approve us buying, a dedicated Mac and an Apple ID for the assistant?
4. Would the client accept a hosted iMessage relay, given that every executive message would pass through that vendor?
5. Should the assistant answer in group chats, or only in direct messages for the MVP?

Risks

Risk	Impact	Mitigation
No Mac available	Cannot test a real iMessage channel	Simulator covers development; borrow a team Mac for the demo
Apple account restrictions on automated use	The assistant's Apple ID could be limited (observed: a new Apple ID could not activate iMessage; see Deployment log)	Dedicated Apple ID, low volume, human-paced replies; confirm Apple's terms with the client
SSH topology send failures (AppleEvents -1743)	Inbound works but replies fail	Test a real send through the wrapper first, or run <code>imgsg</code> in the logged-in user session
Advanced features need SIP disabled	No tapbacks, edits or polls	Keep MVP to text + attachments, which work without it
Hosted relay data exposure	Executive messages processed by a third party	Only with client approval and a reviewed security report

Sources and evidence grades

Grades: **S** = official documentation stating the fact directly; **A** = official source, one step of inference; **B** = community or issue-tracker report; **C** = vendor marketing, not independently verified. Checked on 2026-09-27.

Source	Used for	Grade
OpenClaw: iMessage channel	Official plugin, <code>imgsg</code> JSON-RPC over stdio, restart recovery	S
OpenClaw: iMessage setup	Mac requirements, permissions, SSH topology, -1743 issue, SIP for private API	S
OpenClaw: iMessage media	<code>includeAttachments</code> default, 16 MB media cap, 4,000-character chunks, addressing	S
OpenClaw: BlueBubbles removal	<code>imgsg</code> is the only supported iMessage path	S

Source	Used for	Grade
OpenClaw: building channel plugins	What a channel plugin owns vs core	S
OpenClaw issue #44406	Hosted relay approach for cloud-hosted agents	B
openclaw-linq-plugin	Community plugin for a hosted iMessage API	B
Linq iMessage API	Vendor capabilities and compliance claims	C
TechCrunch on Linq	Linq's product history; contrast with Apple Messages for Business	A
9/21 sponsor meeting transcript	Aarvin's two options, VG runs on a core VM, Gmail intake adapter	S

Contamination note. Most comparisons of iMessage API providers online are published by the vendors themselves, including Linq; treat any vendor-to-vendor comparison as C until checked.